



Chapter six

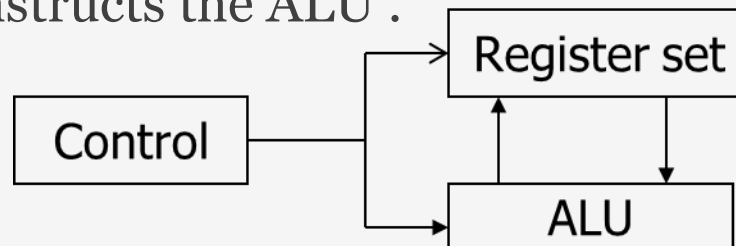
Central Processing Unit

Outline

- General register organization
- Stack organization
- Instruction formats
- Addressing modes
- Data transfer and manipulation
- Program control
- Characteristics of RISC and CISC

What is central processing unit(CPU) ?

- ❖ it is the part of computer that performs the bulk of data-processing operations
- ❖ The CPU is made up of three major parts,
 - The register set :-stores intermediate data used during the execution of the instructions.
 - The arithmetic logic unit (ALU) :-performs the required microoperations for executing the instructions.
 - The control unit :-supervises the transfer of information among the registers and instructs the ALU .

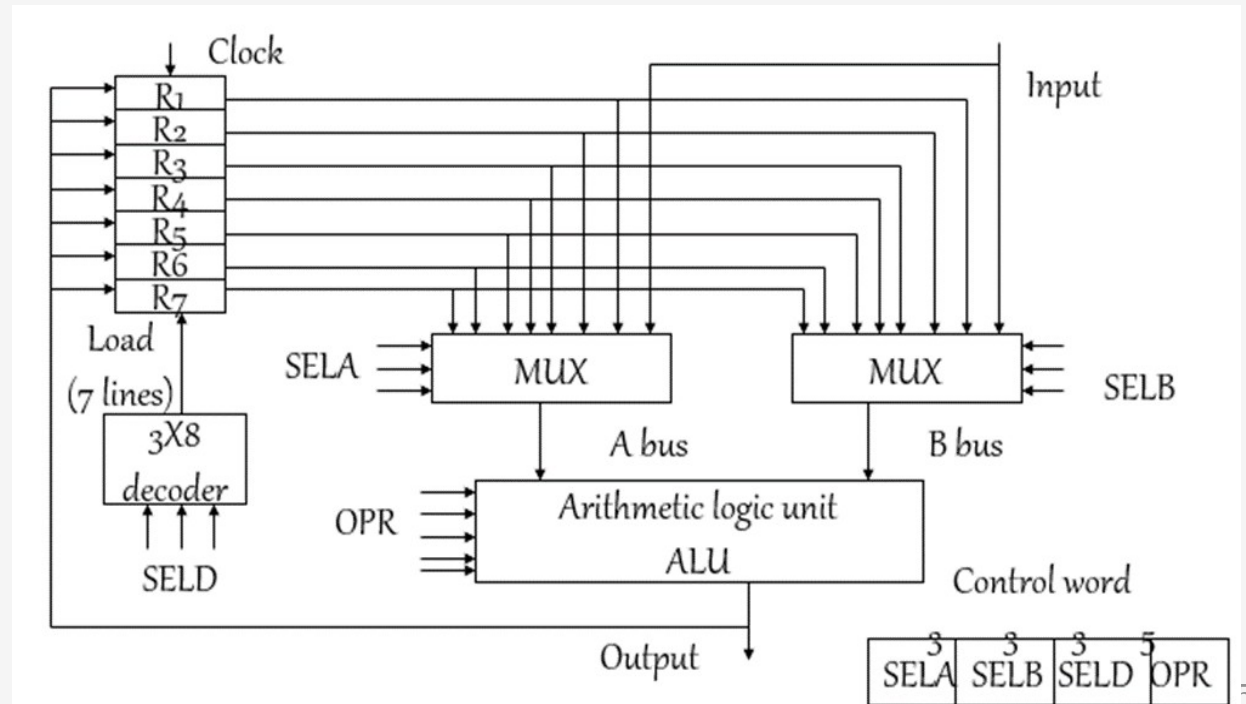


General Register Organization

- ❖ Memory locations are needed for storing pointers, counters, return addresses, temporary results, and partial products during multiplication.
- ❖ Having to refer to memory locations for such applications is time consuming
- ❖ It is more convenient and more efficient to store these intermediate values in processor registers.
- ❖ When a large number of registers are included in the CPU, it is most efficient to connect them through a common bus system.
- ❖ The registers communicate with each other not only for direct data transfers, but also while performing various microoperations.

Cont..

- ❖ A bus organization for seven CPU registers is shown in Fig. 2.
- ❖ The output of each register is connected to two multiplexers (MUX) to form the two buses A and B.
- ❖ The selection lines in each multiplexer select one register or the input data for the particular bus.



Cont..

- ❖ The A and B buses form the inputs to a common arithmetic logic unit (ALU).
- ❖ The operation selected in the ALU determines the arithmetic or logic micro-operation that is to be performed.
- ❖ The result of the microoperation is available for output data and also goes into the inputs of all the registers.
- ❖ The register that receives the information from the output bus is selected by a decoder.
- ❖ The decoder activates one of the register load inputs, thus providing a transfer path between the data in the output bus and the inputs of the selected destination register.

Cont..

- ❖ The control unit that operates the CPU bus system directs the information flow through the registers and ALU by selecting the various components in the system.
- ❖ For example, to perform the operation
$$R_1 \leftarrow R_2 + R_3$$
- ❖ The control must provide binary selection variables to the following selector inputs:
 1. MUX A selector (SELA): to place the content of R2 into bus A.
 2. MUX B selector (SELB): to place the content of R 3 into bus B.
 3. ALU operation selector (OPR): to provide the arithmetic addition $A + B$.
 4. Decoder destination selector (SELD): to transfer the content of the output bus into R1.

Control Word

- ❖ The four control selection variables are generated in the *control unit* and must be available at the beginning of a clock cycle.
- ❖ There are 14 binary selection inputs in the unit
 - ✓ The three bits of SELA select a source register for the A input of the ALU.
 - ✓ The three bits of SELB select a register for the B input of the ALU.
 - ✓ The three bits of SELD select a destination register using the decoder and its seven load outputs.
 - ✓ The five bits of OPR select one of the operations in the ALU.

Cont..

❖ The encoding of the register selections is specified in Table 1.

TABLE 1 Encoding of Register Selection Fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

Cont..

- ❖ The encoding of the ALU operations for the CPU is specified in Table 2.
- ❖ The OPR field has five bits and each operation is designated with a symbolic name.

TABLE 2 Encoding of ALU Operations

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	Add $A + B$	ADD
00101	Subtract $A - B$	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

Stack Organization:

- ❖ A stack or last-in first-out (LIFO) is useful feature that is included in the CPU of most computers.
- ❖ Stack:
 - ✓ A stack is a storage device that stores information in such a manner that the item stored last is the first item retrieved.
- ❖ The operation of a stack can be compared to a stack of trays. The last tray placed on top of the stack is the first to be taken off.
- ❖ In the computer stack is a memory unit with an address register that can count the address only.
- ❖ The register that holds the address for the stack is called a stack pointer (SP). It always points at the top item in the stack.
- ❖ The two operations that are performed on stack are the ***insertion and deletion.***

Cont..

- ❖ A stack can be placed in a portion of a large memory or it can be organized as a collection of a finite number of memory words or registers.
- ❖ The below figure shows the organization of a 64-word register stack

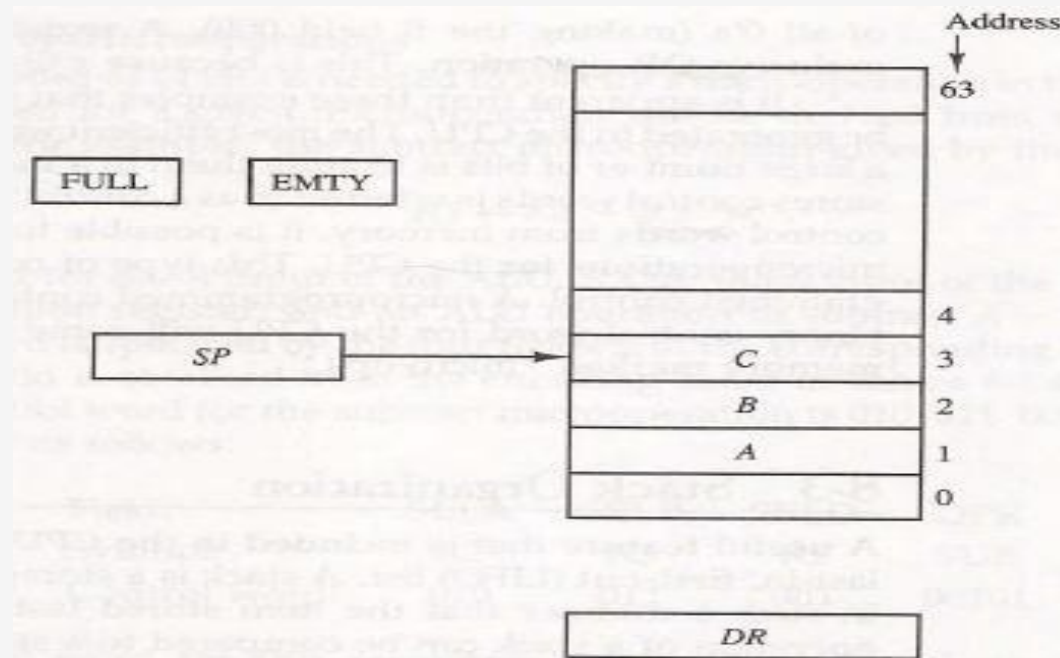


Figure 8-3 Block diagram of a 64-word stack.

Cont..

PUSH:

- ❖ The operation of insertion is called *PUSH*.
- ❖ The push operation is implemented with the following sequence of microoperations:
 - ❖ The stack pointer is incremented so that it points to the address of next-higher word.
 - ❖ A memory write operation inserts the word from DR the top of the stack.
 - ❖ The first item stored in the stack is at address 1.
 - ❖ The last item is stored at address 0.
 - ❖ If SP reaches 0, the stack is full of items, so FULL is to 1.

Cont..

- ❖ This condition is reached if the top item prior to the last push was location 63 and, after incrementing SP, the last item is stored in location 0.
- ❖ Once an item is stored in location 0, there are no more empty registers in the stack, so the EMPT is cleared to 0.

$SP \leftarrow SP + 1$	Increment stack pointer
$M[SP] \leftarrow DR$	Write item on top of the stack
If $(SP = 0)$ then $(FULL \leftarrow 1)$	Check if stack is full
$EMTY \leftarrow 0$	Mark the stack not empty

Cont..

❖POP:

❖The operation of deletion is called *POP*.

❖A new item is deleted from the stack if the stack is not empty (if $EMPTY \neq 0$).

❖The pop operation consists of the following sequence of min operations:

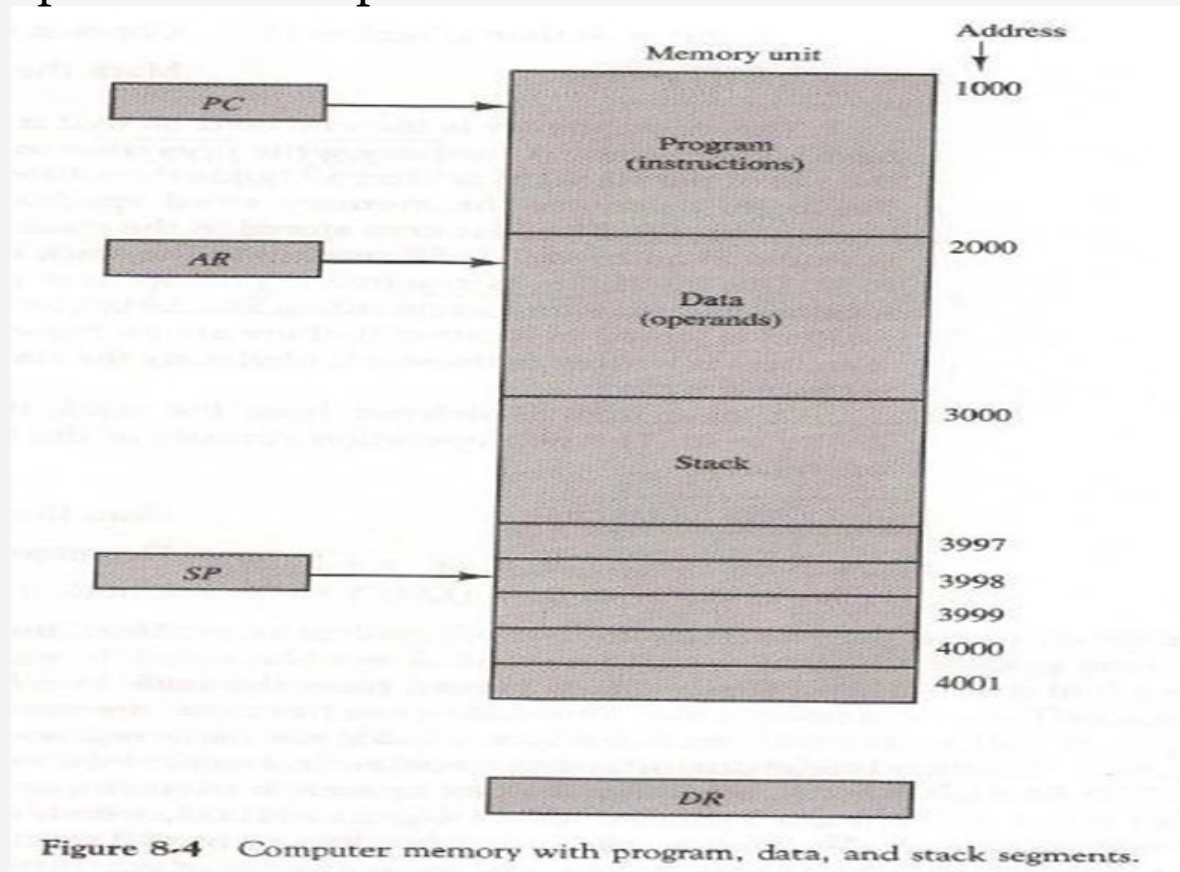
$DR \leftarrow M[SP]$	Read item from the top of stack
$SP \leftarrow SP - 1$	Decrement stack pointer
If $(SP = 0)$ then $(EMPTY \leftarrow 1)$	Check if stack is empty
$FULL \leftarrow 0$	Mark the stack not full

Cont..

- ❖ The top item is read from the stack into DR.
- ❖ The stack pointer is then decremented. If its value reaches zero, the stack is empty, so EMTY is set 1.
- ❖ This condition is reached if the item read was in location 1. Once this it is read out, SP is decremented and reaches the value 0, which is the initial value of SP.
- ❖ If a pop operation reads the item from location 0 and then is decremented, SP changes to 111111, which is equivalent to decimal 63 in above configuration, the word in address 0 receives the last item in the stack.

Memory Stack:

- ❖ PC points at the address of the next instruction in program.
- ❖ The address register AR points at an array of data.
- ❖ The stack pointer SP points at the top of the stack.



Memory Stack:

- ❖ The three registers are connected to a common address bus, and either one can provide an address for memory.
 - ✓ PC is used during the fetch phase to read an instruction.
 - ✓ AR is used during the exec phase to read an operand.
 - ✓ SP is used to push or pop items into or from stack.
- ❖ As shown in Fig. 8-4, the initial value of SP is 4001 and the stack grows with decreasing addresses.
- ❖ Thus the first item stored in the stack is at address 4000, the second item is stored at address 3999, and the last address that can be used for the stack is 3000.

Cont..

- The items in the stack communicate with a data register *DR*. A new item is inserted with the push operation as follows:

$$SP \leftarrow SP-1$$

$$M[SP] \leftarrow DR$$

- The stack pointer is decremented so that it points at the address of the next word.
- A memory write operation inserts the word from *DR* into the top of stack. A new item is deleted with a pop operation as follows:

$$DR \leftarrow M[SP]$$

$$SP \leftarrow SP+1$$

- The top item is read from the stack into *DR*. The stack pointer is then decremented to point at the next item in the stack.

Reverse polish notation

- A stack organization is very effective for evaluating arithmetic expressions.
- The common mathematical method of writing arithmetic expressions imposes difficulties when evaluated by a computer. The common arithmetic expressions are written in infix notation, with each operator written between the operands. Consider the simple arithmetic expression.

$A + B$

- The postfix notation, referred to as reverse Polish notation (RPN), places the operator after the operands. The following examples demonstrate the three representations: $A + B$ Infix notation $+AB$ Prefix or Polish notation $AB +$ Postfix or reverse Polish notation The reverse Polish notation is in a form suitable for stack manipulation.

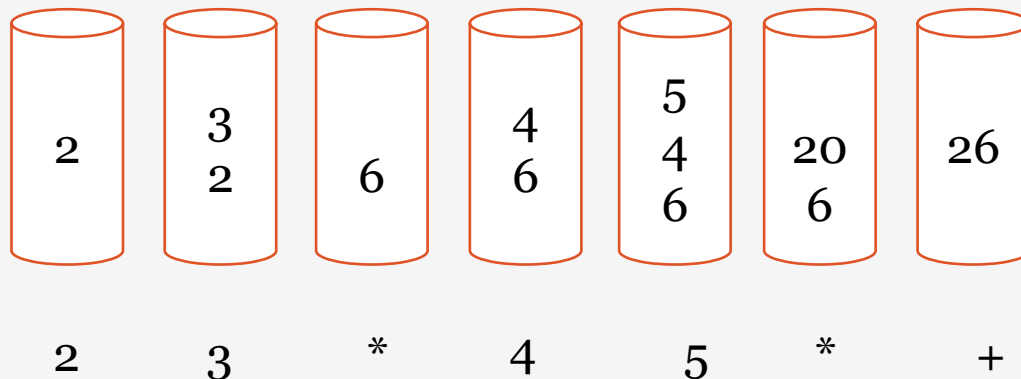
Reverse polish notation

➤ RPN conversion

Example $(A*B) + (C*D) = AB*CD*+$

➤ RPN evaluation

$$(2*3) + (4*5) = 26$$



exercise

$$(A + B) * [C * (D + E) + F] = AB + DE + C * F + *$$

Specify the control word that must be applied to the processor of Fig. 8-2 to implement the following microoperations.

- a. $R1 \leftarrow R2 + R3$
- b. $R4 \leftarrow R4$
- c. $R5 \leftarrow R5 - 1$
- d. $R6 \leftarrow \text{shl } R1$
- e. $R7 \leftarrow \text{input}$

Determine the microoperations that will be executed in the processor of Fig. 8-2 when the following 14-bit control words are applied.

- a. 00101001100101
- b. 00000000000000
- c. 01001001001100
- d. 00000100000010
- e. 11110001110000

exercise

Convert the following arithmetic expressions from infix to reverse Polish notation.

a. $A * B + C * D + E * F$

b. $A * B + A * (B * D + C * E)$

c. $A + B * [C * D + E * (F + G)]$

d.
$$\frac{A * [B + C * (D + E)]}{F * (G + H)}$$

Convert the following numerical arithmetic expression into reverse Polish notation and show the stack operations for evaluating the numerical result.

$$(3 + 4)[10(2 + 6) + 8]$$

Part - II

2. Instruction Formats:

- ❖ The format of an instruction is usually depicted in a rectangular box symbolizing .
- ❖ The bits of the instruction are divided into groups called fields.
- ❖ The most common fields found in instruction formats are:
 1. An operation code field that specifies the operation to be performed
 2. An address field that designates a memory address or a processor register.
 3. A mode field that specifies the way the operand or the effective address is determined.
- ❖ The number of address fields in the instruction format of a computer depends on the internal organization of its registers.

Cont..

❖ Most computers fall into one of three types of CPU organizations:

1. Single accumulator organization.
2. General register organization.
3. Stack organization.

Single Accumulator Organization:

- ✓ The instruction format in this type of computer uses one address field.
- ✓ For example, the instruction that specifies an arithmetic addition defined by an assembly language instruction as
 - **ADD X**
- ✓ Where X is the address of the operand. The ADD instruction in this case results in the operation $AC \leftarrow AC + M[X]$. AC is the accumulator register and $M[X]$ symbolizes the memory word located at address X.

Cont..

General register organization:

- ✓ The instruction format in this type of computer needs three register address fields.
- ✓ Thus the instruction for an arithmetic addition may be written in an assembly language as
 - **ADD R1, R2, R3**
to denote operation $R1 \leftarrow R2 + R3$. The number of address fields in the instruction can be reduced from three to two if the destination register is the same as one of the source registers.
- ✓ Thus the instruction **ADD R1, R2** would denote the operation $R1 \leftarrow R1 + R2$. Only register addresses for R1 and R2 need be specified in this instruction.
- ✓ General register-type computers employ two or three address fields in their instruction format.
- ✓ Each address field may specify a processor register or a memory word.
- ✓ An instruction symbolized by **ADD R1, X** would specify the operation $R1 \leftarrow R1 + M[X]$.

It has two address fields, one for register R1 and the other for the memory address X

Stack organization:

- ✓ The stack-organized CPU has PUSH and POP instructions which require an address field.
- ✓ Thus the instruction **PUSH X** will push the word at address X to the top of the stack.
- ✓ The stack pointer is updated automatically.
- ✓ Operation-type instructions do not need an address field in stack-organized computers.
- ✓ This is because the operation is performed on the two items that are on top of the stack.
- ✓ The instruction **ADD** in a stack computer consists of an operation code only with no address field.
- ✓ This operation has the effect of popping the two top numbers from the stack, adding the numbers, and pushing the sum into the stack.
- ✓ There is no need to specify operands with an address field since all operands are implied to be in the stack.

❖ Three Address Instructions:

- ❖ Three-address instruction formats can use each address field to specify either a processor register or a memory operand.
- ❖ The program assembly language that evaluates $X = (A+B) * (C+D)$ is shown below, together with comments that explain the register transfer operation of each instruction.

```
ADD    R1, A, B    R1 ← M[A] + M[B]
ADD    R2, C, D    R2 ← M[C] + M[D]
MUL    X, R1, R2   M[X] ← R1 * R2
```

- ❖ The symbol $M[A]$ denotes the operand at memory address symbolized by A.
- ❖ The advantage of the three-address format is that it results in short programs when evaluating arithmetic expressions.
- ❖ The disadvantage is that the binary-coded instructions require too many bits to specify three addresses.

❖ Two Address Instructions:

❖ Two-address instructions formats use each address field can specify either a processor register or memory word.

❖ The program to evaluate $X = (A+B) * (C+D)$ is as follows

MOV	R1, A	$R1 \leftarrow M[A]$
ADD	R1, B	$R1 \leftarrow R1 + M[B]$
MOV	R2, C	$R2 \leftarrow M[C]$
ADD	R2, D	$R2 \leftarrow R2 + M[D]$
MUL	R1, R2	$R1 \leftarrow R1 * R2$
MOV	X, R1	$M[X] \leftarrow R1$

❖ The MOV instruction moves or transfers the operands to and from memory and processor registers.

❖ The first symbol listed in an instruction is assumed be both a source and the destination where the result of the operation transferred.

One Address Instructions:

- ❖ One-address instructions use an implied accumulator (AC) register for all data manipulation.
- ❖ For multiplication and division there is a need for a second register. But for the basic discussion we will neglect the second register and assume that the AC contains the result of all operations.
- ❖ The program to evaluate $X = (A+B) * (C+D)$ is

LOAD	A	$AC \leftarrow M[A]$
ADD	B	$AC \leftarrow AC + M[B]$
STORE	T	$M[T] \leftarrow AC$
LOAD	C	$AC \leftarrow M[C]$
ADD	D	$AC \leftarrow AC + M[D]$
MUL	T	$AC \leftarrow AC * M[T]$
STORE	X	$M[X] \leftarrow AC$

- ❖ All operations are done between the AC register and a memory operand.
- ❖ T is the address of a temporary memory location required for storing the intermediate result.

Zero Address Instructions:

- ❖ A stack-organized computer does not use an address field for the instructions ADD and MUL.
- ❖ The PUSH and POP instructions, however, need an address field to specify the operand that communicates with the stack.
- ❖ The following program shows how $X = (A+B) * (C+D)$ will be written for a stack-organized computer.
- ❖ (TOS stands for top of stack).

```
PUSH    A    TOS ← A
PUSH    B    TOS ← B
ADD      TOS ← ( A + B )
PUSH    C    TOS ← C
PUSH    D    TOS ← D
ADD      TOS ← ( C + D )
MUL      TOS ← ( C + D ) * ( A + B )
POP      X    M[X] ← TOS
```

- ❖ To evaluate arithmetic expressions in a stack computer, it is necessary to convert the expression into reverse Polish notation.
- ❖ The name "zero-address" is given to this type of computer because of the absence of an address field in the computational instructions.

Addressing Modes

- ❖ The way the operands are chosen during program execution is dependent on the addressing mode of the instruction.
- ❖ Computers use addressing mode techniques for the purpose of accommodating one or both of the following provisions:
 - ✓ To give programming versatility to the user by providing such facilities as pointers to memory, counters for loop control, indexing of data, and program relocation.
 - ✓ To reduce the number of bits in the addressing field of the instruction
- ❖ Most addressing modes modify the address field of the instruction; there are two modes that need no address field at all. These are *implied* and *immediate* modes.

Implied Mode:

- ✓ In this mode the operands are specified implicitly in the definition of the instruction.
- ✓ For example, the instruction "complement accumulator" is an implied-mode instruction because the operand in the accumulator register is implied in the definition of the instruction.
- ✓ All register reference instructions that use an accumulator are implied mode instructions.
- ✓ Zero address in a stack organization computer is implied mode instructions.

Immediate Mode:

- ✓ In this mode the operand is specified in the instruction itself.
- ✓ In other words an immediate-mode instruction has an operand rather than an address field.

❖ **Register Mode:**

- ❖ In this mode the operands are in registers that reside within the CPU.
- ❖ The particular register is selected from a register field in the instruction.

Register Indirect Mode:

- ❖ In this mode the instruction specifies a register in CPU whose contents give the address of the operand in memory.
- ❖ In other words, the selected register contains the address of the operand rather than the operand itself.
- ❖ The advantage of a register indirect mode instruction is that the address field of the instruction uses few bits to select a register than would have been required to specify a memory address directly.

Auto-increment or Auto-Decrement Mode:

- ✓ This is similar to the register indirect mode except that the register is incremented or decremented after (or before) its value is used to access memory.
- The address field of an instruction is used by the control unit in the CPU to obtain the operand from memory.
- Sometimes the value given in the address field is the address of the operand, but sometimes it is just an address from which the address of the operand is calculated.
- The basic two mode of addressing used in CPU are *direct* and *indirect* address mode.

Direct Address Mode:

- ✓ In this mode the effective address is equal to the address part of the instruction.
- ✓ The operand resides in memory and its address is given directly by the address field of the instruction.
- ✓ In a branch-type instruction the address field specifies the actual branch address.

Indirect Address Mode:

- ✓ In this mode the address field of the instruction gives the address where the effective address is stored in memory.
- ✓ Control fetches the instruction from memory and uses its address part to access memory again to read the effective address.
- A few addressing modes require that the address field of the instruction be added to the content of a specific register in the CPU.
- The effective address in these modes is obtained from the following computation:

$$\text{Effective address} = \text{address part of instruction} + \text{content of CPU register}$$

- The CPU register used in the computation may be the program counter, an index register, or a base register.
- We have a different addressing mode which is used for a different application.

Relative Address Mode:

- ✓ In this mode the content of the program counter is added to the address part of the instruction in order to obtain the effective address.

Indexed Addressing Mode:

- ✓ In this mode the content of an index register is added to the address part of the instruction to obtain the effective address.
- ✓ An index register is a special CPU register that contains an index value.

Base Register Addressing Mode:

- ✓ In this mode the content of a base register is added to the address part of the instruction to obtain the effective address.
- ✓ This is similar to the indexed addressing mode except that the register is now called a base register instead of an index register.

Data Transfer and Manipulation:

➤ Most computer instructions can be classified into three categories:

1. *Data transfer instructions*
2. *Data manipulation instructions*
3. *Program control instructions*

Data Transfer Instructions:

- Data transfer instructions move data from one place in the computer to another without changing the data content.
- The most common transfers are between memory and processor registers, between processor registers and input or output, and between the processor registers themselves.

Table 8-5 gives a list of eight data transfer instructions used in many computers.

TABLE 8-5 Typical Data Transfer Instructions

Name	Mnemonic
Load	LD
Store	ST
Move	MOV
Exchange	XCH
Input	IN
Output	OUT
Push	PUSH
Pop	POP

Data Manipulation Instructions:

- Data manipulation instructions perform operations on data and provide the computational capabilities for the computer.
- The data manipulation instructions in a typical computer are usually divided into three basic types:
 1. Arithmetic instructions
 2. Logical and bit manipulation instructions
 3. Shift instructions

✓ **Arithmetic instructions**

- ✓ The four basic arithmetic operations are addition, subtraction, multiplication and division.
- ✓ Most computers provide instructions for all four operations.
- ✓ Some small computers have only addition and possibly subtraction instructions. The multiplication and division must then be generated by mean software subroutines.

A list of typical arithmetic instructions is given in Table 8-

TABLE 8-7 Typical Arithmetic Instructions

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate (2's complement)	NEG

1 Logical and bit manipulation instructions

- ❖ Logical instructions perform binary operations on strings of bits store, registers.
- ❖ They are useful for manipulating individual bits or a group of that represent binary-coded information.
- ❖ The logical instructions consider each bit of the operand separately and treat it as a Boolean variable.
- ❖ By proper application of the logical instructions it is possible to change bit values, to clear a group of bits, or to insert new bit values into operands stored in register memory words.

❖ Some typical logical and bit manipulation instructions are listed in Table 8-8.

TABLE 8-8 Typical Logical and Bit Manipulation Instructions

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interrupt	EI
Disable interrupt	DI

Shift Instructions:

- ✓ Shifts are operations in which the bits of a word are moved to the left or right.
- ✓ The bit shifted in at the end of the word determines the type of shift used.
- ✓ Shift instructions may specify logical shifts, arithmetic shifts, or rotate-type operations.
- ✓ In either case the shift may be to the right or to the left.
- ✓ Table 8-9 lists four types of shift instructions.

Cont

TABLE 8-9 Typical Shift Instructions

Name	Mnemonic
Logical shift right	SHR
Logical shift left	SHL
Arithmetic shift right	SHRA
Arithmetic shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC

Program Control:

- ❖ Program control instructions specify conditions for altering the content of the program counter.
- ❖ The change in value of the program counter as a result of the execution of a program control instruction causes a break in the sequence of instruction execution.
- ❖ This instruction provides control over the flow of program execution and a capability for branching to different program segments.
- ❖ Some typical program control instructions are listed in Table 8.10.

TABLE 8-10 Typical Program Control Instructions

Name	Mnemonic
Branch	BR
Jump	JMP
Skip	SKP
Call	CALL
Return	RET
Compare (by subtraction)	CMP
Test (by ANDing)	TST

-
- ❖ Branch and jump instructions may be conditional or unconditional.
 - ❖ An unconditional branch instruction causes a branch to the specified address without any conditions.
 - ❖ The conditional branch instruction specifies a condition such as branch if positive or branch if zero.

Program Interrupt:

- ❖ Program interrupt refers to the transfer of program control from a currently running program to another service program as a result of an external or internal generated request.
- ❖ The interrupt procedure is similar to a subroutine call except for three variations:
 - ✓ The interrupt is initiated by an internal or external signal.
 - ✓ Address of the interrupt service program is determined by the hardware.
 - ✓ An interrupt procedure usually stores all the information rather than storing only PC content.

Types of interrupts:

- ❖ There are three major types of interrupts that cause a break in the normal execution of a program.
- ❖ They can be classified as

External interrupts:

- ✓ These come from input—output (I/O) devices, from a timing device, from a circuit monitoring the power supply, or from any other external source.
- ✓ Ex: I/O device requesting transfer of data, I/O device finished transfer of data, elapsed time of an event, or power failure.

Internal interrupts:

- ✓ These arise from illegal or erroneous use of an instruction or data.
- ✓ Internal interrupts are also called traps.
- ✓ Ex: interrupts caused by internal error conditions are register overflow, attempt to divide by zero, an invalid operation code, stack overflow, and protection violation.
- ❖ Internal and external interrupts are initiated from signals that occur in hardware of CPU.

Software interrupts

- ✓ A software interrupt is initiated by executing an instruction.
- ✓ Software interrupt is a special call instruction that behaves like an interrupt rather than a subroutine call.

Reduced Instruction Set Computer:

- ❖ A computer with large number instructions is classified as a *complex instruction set computer*, abbreviated as CISC.
- ❖ The computer which having the fewer instructions is classified as a *reduced instruction set computer*, abbreviated as RISC.

CISC Characteristics:

- ✓ A large number of instructions--typically from 100 to 250 instructions.
- ✓ Some instructions that perform specialized tasks and are used infrequently.
- ✓ A large variety of addressing modes—typically from 5 to 20 differ modes.
- ✓ Variable-length instruction formats
- ✓ Instructions that manipulate operands in memory

RISC Characteristics:

- ✓ Relatively few instructions
- ✓ Relatively few addressing modes
- ✓ Memory access limited to load and store instructions
- ✓ All operations done within the registers of the CPU
- ✓ Fixed-length, easily decoded instruction format
- ✓ Single-cycle instruction execution
- ✓ Hardwired rather than microprogrammed control
- ✓ A relatively large number of registers in the processor unit
- ✓ Efficient instruction pipeline

?